

Sistema de gestión de propiedades inmobiliarias con Django y Ollama

Portada

- **Título:** Sistema de gestión de propiedades inmobiliarias con Django y Ollama
 - **Estudiante:** Christian Guizada Cuellar
 - **Asignatura:** Programación IV
 - **Docente:** Jared Lopez
 - **Fecha:** 28 de septiembre de 2026
-

Introducción

Este informe documenta el desarrollo del Entregable Final de Programación IV. El proyecto consiste en un sistema web desarrollado con Django para administrar propiedades inmobiliarias mediante un CRUD completo, generar reportes y realizar consultas mediante un chatbot conectado a una instancia local de Ollama.

El proyecto utiliza el modelo personalizado `casas-qwen2.5`, basado en `qwen2.5:1.5b`. Django prepara el contexto utilizando información real almacenada en la base de datos y posteriormente envía los datos relevantes al servicio local de Ollama.

También se implementaron validaciones, manejo de errores, cinco reportes, dos patrones de diseño, pruebas unitarias, historial persistente del chat y documentación de las sesiones realizadas con OpenCode.

Todo lo descrito en este informe corresponde al estado final del proyecto.

Punto 1: Diseño e implementación del CRUD

1.1 Entidad y modelo de datos

La entidad principal seleccionada para el sistema es una **propiedad inmobiliaria**, representada mediante el modelo `House`.

El modelo cuenta con 7 campos principales:

```
class House(models.Model):
    codigo = models.CharField(max_length=20, unique=True, default="SIN-
CODIGO")
    name = models.CharField(max_length=200)
    location = models.CharField(max_length=200)
    price = models.DecimalField(
```

```
        max_digits=12,  
        decimal_places=2,  
        validators=[MinValueValidator(Decimal('0'))]  
    )  
    description = models.TextField()  
    dormitorios = models.IntegerField(default=1)
```

Los campos principales son:

- **codigo**: identificador único de la propiedad.
- **name**: nombre de la propiedad.
- **location**: ubicación.
- **price**: precio de la propiedad.
- **description**: descripción.
- **dormitorios**: cantidad de dormitorios.
- **id**: identificador generado automáticamente por Django.

Además, el proyecto conserva entidades relacionadas:

- **Place**: representa lugares cercanos.
- **HouseDistance**: relaciona propiedades con lugares cercanos y almacena la distancia.
- **ChatMessage**: almacena las preguntas y respuestas realizadas en el chat.

El precio utiliza **DecimalField**, lo que permite realizar comparaciones y cálculos correctamente.

1.2 Migraciones

Las migraciones se realizaron mediante los comandos:

```
python manage.py makemigrations chatbot  
python manage.py migrate
```

Las principales migraciones existentes son:

```
0001_initial.py  
0002_alter_place_category_chatmessage.py  
0003_house_codigo_house_dormitorios.py  
0004_alter_house_price.py
```

La última migración convirtió el campo **price** de texto a **DecimalField**, realizando previamente una limpieza de los precios existentes.

El proyecto también cuenta con un panel de administración personalizado en:

```
chatbot/admin.py
```

En el administrador se configuraron listas, búsquedas y filtros para las entidades principales.

1.3 Vistas y rutas del CRUD

El CRUD se implementa mediante vistas de Django y `HouseForm`.

Las principales rutas son:

```
urlpatterns = [
    path('', views.index, name='index'),
    path('propiedades/nueva/', views.house_create, name='house_create'),
    path('propiedades/<int:house_id>/', views.house_detail,
name='house_detail'),
    path('propiedades/<int:house_id>/editar/', views.house_update,
name='house_update'),
    path('propiedades/<int:house_id>/eliminar/', views.house_delete,
name='house_delete'),
    path('reportes/', views.reportes_view, name='reportes'),
    path('chat/<int:house_id>/', views.chat_view, name='chat'),
]
```

El sistema permite:

- visualizar el catálogo de propiedades;
- consultar el detalle de una propiedad;
- crear una propiedad;
- editar una propiedad;
- eliminar una propiedad;
- consultar cinco reportes;
- abrir el chat de una propiedad.

Las operaciones de creación, edición y eliminación utilizan `HouseForm` y mensajes de éxito o error mediante `django.contrib.messages`.

1.4 Validaciones

Se implementaron validaciones para evitar datos incorrectos.

Código único

El campo `codigo` es obligatorio y único:

```
codigo = models.CharField(
    max_length=20,
    unique=True,
    default="SIN-CODIGO"
)
```

Además, el formulario comprueba que no esté vacío.

Precio

El precio no puede ser negativo:

```
def clean_price(self):
    price = self.cleaned_data['price']

    if price is None:
        raise forms.ValidationError("El precio es obligatorio.")

    if price < 0:
        raise forms.ValidationError(
            "El precio no puede ser negativo."
        )

    return price
```

Dormitorios

La cantidad de dormitorios tampoco puede ser negativa:

```
def clean_dormitorios(self):
    dormitorios = self.cleaned_data['dormitorios']

    if dormitorios is None or dormitorios < 0:
        raise forms.ValidationError(
            "Los dormitorios no pueden ser negativos."
        )

    return dormitorios
```

Estas validaciones permiten rechazar códigos repetidos, precios negativos, cantidades de dormitorios negativas y datos inválidos.

1.5 Reportes

El sistema cuenta con cinco reportes disponibles desde:

```
/reportes/
```

Los reportes utilizan datos reales almacenados en la base de datos.

Los cinco reportes son:

1. Total de propiedades.
2. Propiedad más cara.
3. Propiedad más barata.
4. Propiedades con precio inferior al promedio.
5. Propiedades agrupadas por ubicación.

Con los datos de ejemplo actuales se obtuvieron los siguientes resultados:

```
Total de propiedades registradas
=> 3 propiedades registradas.

Propiedad más cara
=> Ático Mirador Real (CASA-002) con precio $650,000.

Propiedad más barata
=> Residencial Los Robles (CASA-003) con precio $320,000.

Propiedades con precio inferior al promedio
=> Promedio: $483,333.
    Por debajo: Villa Serena, Residencial Los Robles.

Propiedades agrupadas por ubicación
=> 3 ubicaciones.
    Av. Las Palmeras 240, Costa Azul: 1
    Paseo de la Castellana 112, Zona Centro: 1
    Calle El Robledal 45, Bosque Real: 1
```

Los datos utilizados corresponden a:

```
Villa Serena          => $480,000
Ático Mirador Real    => $650,000
Residencial Los Robles => $320,000
```

Punto 2: Integración con IA local mediante Ollama

2.1 Instalación y configuración de Ollama

La aplicación utiliza Ollama de manera local.

La versión utilizada durante las pruebas fue:

```
ollama --version
```

Resultado:

```
ollama version is 0.34.0
```

Los modelos disponibles fueron comprobados mediante:

```
ollama list
```

Resultado final:

NAME	ID	SIZE
casas-qwen2.5:latest	13c8ee136d31	986 MB
gemma4:cloud	ef09f235533c	-
qwen2.5:1.5b	65ec06548149	986 MB

El proyecto utiliza específicamente:

```
casas-qwen2.5
```

Este modelo es local y está basado en:

```
qwen2.5:1.5b
```

El modelo personalizado se define en:

```
modelfile-casas
```

Para crear el modelo se utilizan:

```
ollama pull qwen2.5:1.5b  
ollama create casas-qwen2.5 -f modelfile-casas
```

La aplicación se comunica con Ollama mediante:

```
http://localhost:11434
```

2.2 Variables de entorno

La configuración se encuentra en `.env.example`:

```
OLLAMA_URL=http://localhost:11434
OLLAMA_MODEL=casas-qwen2.5
SECRET_KEY=change-this-secret-key
DEBUG=True
```

Esto permite cambiar la configuración sin modificar directamente el código fuente.

2.3 Servicio de Ollama

La comunicación con Ollama se encuentra centralizada en:

```
chatbot/services/ollama_service.py
```

El flujo general es:

```
Usuario
  ↓
chat_view
  ↓
detect_intent / validate_data
  ↓
context.py
  ↓
datos reales de la propiedad
  ↓
ollama_service.py
  ↓
Ollama local
  ↓
respuesta
  ↓
ChatMessage
  ↓
Usuario
```

Django no envía solamente la pregunta. También prepara información real de la propiedad, incluyendo:

- código;
- nombre;
- ubicación;
- precio;

- descripción;
- dormitorios;
- lugares cercanos;
- categorías;
- distancias.

Esto permite que la respuesta del modelo se base en información disponible en la base de datos.

2.4 Restricción de las respuestas

La restricción funciona mediante dos niveles.

Primer nivel: modelo Ollama

El archivo `modelfile-casas` contiene instrucciones que limitan el modelo a consultas relacionadas con propiedades inmobiliarias.

El modelo no debe responder consultas sobre:

- política;
- programación;
- matemáticas;
- noticias;
- deportes;
- personas;
- temas generales;
- instrucciones para ignorar sus restricciones.

Segundo nivel: Django

Además del filtro del modelo, Django analiza la intención de la consulta antes de llamar a Ollama.

La lógica se encuentra en:

```
chatbot/logic.py
```

Las consultas fuera del dominio reciben exactamente:

```
Solo puedo responder consultas sobre propiedades inmobiliarias y sus datos.
```

De esta forma, las consultas no relacionadas con propiedades pueden ser bloqueadas directamente por Django sin necesidad de enviarlas a Ollama.

Si un dato no está disponible en el contexto de la propiedad, se utiliza:

```
No tengo ese dato disponible.
```

Esto evita que el sistema invente información.

2.5 Pruebas reales del chat

Durante las pruebas manuales se realizaron diferentes consultas.

Consulta sobre lugares cercanos

Se realizó:

```
¿Qué lugares cercanos tiene esta propiedad?
```

El sistema respondió utilizando los datos registrados:

```
Lugares cercanos:  
- Centro Médico San Lucas: 450m (Servicios Médicos)  
- Colegio Bilingüe Los Pinos: 750m (Colegios y Educación)  
- Estación de Metro Central: 300m (Transporte)  
- Centro Comercial Gran Plaza: 900m (Centros Comerciales y Tiendas)
```

Esto demuestra que el contexto enviado al modelo contiene información real de lugares cercanos y sus distancias.

Consulta fuera del dominio

Se realizó:

```
cual es la capital de francia?
```

El sistema respondió:

```
Solo puedo responder consultas sobre propiedades inmobiliarias y sus datos.
```

Esto demuestra el funcionamiento de la restricción implementada en Django.

Consulta de información no disponible

Se probó una consulta sobre información que no forma parte de los datos de la propiedad:

¿Cuál es el número de teléfono del propietario?

El sistema no debe inventar un número y aplica la restricción o el mensaje de información no disponible según la clasificación de la consulta.

2.6 Manejo de Ollama no disponible

También se implementó manejo de errores cuando Ollama no está disponible.

El mensaje utilizado es:

No fue posible comunicarse con el servicio de IA local.
Verifique que Ollama esté ejecutándose.

El sistema no muestra el traceback de Python al usuario.

En `chat_view` también se incorporó registro de errores mediante `logging` y se devuelve un mensaje genérico ante errores internos.

2.7 Historial de conversación

El sistema utiliza el modelo `ChatMessage` para conservar el historial:

```
class ChatMessage(models.Model):
    house = models.ForeignKey(
        House,
        on_delete=models.CASCADE,
        related_name='chat_messages'
    )
    question = models.TextField()
    response = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

Cada pregunta y respuesta queda asociada a la propiedad correspondiente.

En la interfaz se agregó una sección:

Historial de conversación

Esta sección muestra:

- pregunta realizada;
- respuesta de la IA;

- fecha y hora.

Las conversaciones anteriores permanecen visibles al volver a ingresar al chat de la propiedad.

Si no existe historial, se muestra un mensaje indicando que todavía no existen conversaciones.

El historial se utiliza para visualización y persistencia. No se envía automáticamente todo el historial a Ollama como contexto, manteniendo separado el historial de la información real utilizada para generar las respuestas.

Punto 3: Calidad, patrones y pruebas

3.1 Patrón Strategy

El patrón **Strategy** se implementó en:

```
chatbot/reports.py
```

Cada reporte se representa mediante una clase independiente con su método `calcular()`.

Entre las estrategias se encuentran:

```
TotalPropiedades  
PropiedadMasCara  
PropiedadMasBarata  
PropiedadesBajoPromedio  
PropiedadesPorUbicacion
```

La vista utiliza las estrategias sin necesitar conocer la implementación interna de cada reporte.

Esto permite agregar nuevos reportes sin tener que modificar directamente la lógica principal de la vista.

3.2 Patrón Factory

El patrón **Factory** se implementó en:

```
chatbot/report_factory.py
```

La clase `ReporteFactory` permite crear el reporte correspondiente según un tipo:

```
reporte = ReporteFactory.crear("mas_cara")
```

```
reporte.calcular()
```

Los tipos disponibles son:

```
total
mas_cara
mas_barata
bajo_promedio
por_ubicacion
```

Si se solicita un tipo inexistente, la Factory genera un `ValueError`.

La Factory centraliza la creación de los reportes y evita que la vista tenga que conocer directamente todas las clases concretas.

3.3 Pruebas unitarias

Las pruebas se ejecutaron mediante:

```
python manage.py test
```

Resultado final:

```
Found 30 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 30 tests in 0.227s

OK
Destroying test database for alias 'default'...
```

El proyecto cuenta con **30 pruebas y todas pasan correctamente**.

Las pruebas cubren diferentes partes del sistema, incluyendo:

- código único;
- validaciones del formulario;
- reportes;
- Factory;
- restricciones del chat;
- datos faltantes;

- servicio de Ollama mediante mocks;
- historial del chat;
- exportación CSV;
- exportación PDF.

Las pruebas relacionadas con Ollama utilizan mocks, por lo que no dependen de que el servicio esté encendido durante la ejecución de la suite.

También se ejecutó:

```
python manage.py check
```

Resultado:

```
System check identified no issues (0 silenced).
```

Esto confirma que Django no detecta problemas en la configuración del proyecto.

Uso de OpenCode

Durante el desarrollo se utilizó OpenCode como herramienta de apoyo para analizar el proyecto, detectar mejoras, implementar cambios pequeños y revisar las pruebas.

Se realizaron tres sesiones principales y cada una quedó documentada dentro de la carpeta `opencode/`.

Sesión 1: Diagnóstico del proyecto

La primera sesión tuvo como objetivo revisar el proyecto existente antes de realizar cambios.

OpenCode analizó:

- modelo `House`;
- validaciones;
- CRUD;
- reportes;
- integración con Ollama;
- flujo del contexto;
- manejo de errores;
- patrones de diseño;
- pruebas existentes.

Entre las observaciones realizadas se identificaron mejoras como:

- revisar el uso de `csrf_exempt`;
- mejorar el manejo de errores internos;
- evitar mostrar excepciones directamente al usuario;
- agregar pruebas de las operaciones CRUD;
- revisar pequeñas duplicaciones y nombres de variables.

El diagnóstico permitió decidir que no era necesario rehacer la arquitectura, sino realizar mejoras puntuales.

La sesión quedó documentada en:

```
opencode/01_diagnostico.md
```

Sesión 2: Mejoras y pruebas

En la segunda sesión se solicitó a OpenCode implementar las mejoras detectadas en el diagnóstico, manteniendo la estructura existente.

Entre los cambios realizados estuvieron:

- eliminación de `@csrf_exempt` del chat;
- incorporación de logging para errores internos;
- uso de mensajes genéricos para errores del servidor;
- renombrado de una variable local;
- eliminación de una duplicación;
- incorporación de pruebas adicionales.

Después de los cambios se ejecutaron:

```
python manage.py check
python manage.py test
```

El resultado de esta etapa fue:

```
System check identified no issues (0 silenced).
Ran 28 tests ... OK
```

La sesión quedó documentada en:

```
opencode/02_mejoras_y_pruebas.md
```

Sesión 3: Historial de conversación

La tercera sesión tuvo como objetivo agregar un historial persistente al chat y realizar una pequeña mejora visual.

Se indicó a OpenCode reutilizar el modelo existente:

```
class ChatMessage(models.Model):
    house = models.ForeignKey(
        House,
        on_delete=models.CASCADE,
        related_name='chat_messages'
    )
    question = models.TextField()
    response = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

No fue necesario crear otro modelo ni realizar nuevas migraciones.

OpenCode modificó principalmente:

```
chatbot/templates/chatbot/chat.html
chatbot/tests.py
```

La sección visual que anteriormente mostraba las "Ideas de preguntas" pasó a utilizar el espacio para mostrar:

```
Historial de conversación
```

El historial muestra las preguntas, respuestas y fecha/hora de cada conversación.

Las ideas de preguntas se conservaron como pequeños atajos para no eliminar completamente esa funcionalidad.

También se agregaron pruebas para comprobar el historial.

El resultado final fue:

```
python manage.py check
python manage.py test
```

Con el siguiente resultado:

```
System check identified no issues (0 silenced).
Ran 30 tests ... OK
```

La sesión quedó documentada en:

```
opencode/03_historial_chat.md
```

Impacto de OpenCode en el desarrollo

OpenCode fue utilizado como herramienta de apoyo durante el desarrollo y no como sustituto de la comprobación del proyecto.

Sus principales aportes fueron:

- detectar pequeñas mejoras en la implementación;
- ayudar a revisar el manejo de errores;
- aumentar la cobertura de pruebas;
- mejorar la organización del código;
- implementar el historial visual aprovechando el modelo existente;
- documentar las decisiones y cambios realizados.

Cada modificación importante fue comprobada posteriormente mediante los comandos de Django y mediante pruebas manuales.

La evidencia de las sesiones se conserva en:

```
opencode/
├── 01_diagnostico.md
├── 02_mejoras_y_pruebas.md
└── 03_historial_chat.md
```

3.4 Estructura principal del proyecto

La estructura relevante del proyecto es:

```
chatbot/
├── admin.py
├── ai.py
├── apps.py
├── context.py
├── forms.py
├── logic.py
├── models.py
├── pdf_export.py
├── report_factory.py
├── reports.py
├── tests.py
├── urls.py
├── views.py
├── services/
│   └── ollama_service.py
├── templates/
│   └── chatbot/

contextchat/
├── settings.py
├── urls.py
├── asgi.py
└── wsgi.py

opencode/
├── 01_diagnostico.md
├── 02_mejoras_y_pruebas.md
└── 03_historial_chat.md

manage.py
requirements.txt
.env.example
README.md
DOCUMENTACION.md
modelfile-casas
```

3.5 Dependencias

Las dependencias utilizadas se registraron desde el entorno virtual mediante **pip freeze**.

El archivo generado es:

```
requirements.txt
```

Entre las principales dependencias se encuentran:

```
Django==5.2.17  
requests==2.34.2  
python-dotenv==1.2.3  
rich==15.0.0
```

El proyecto utiliza Python 3.11.

3.6 Configuración y ejecución

Para preparar el proyecto se utiliza un entorno virtual:

```
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt
```

Luego se configura el archivo de entorno:

```
cp .env.example .env
```

Se aplican las migraciones:

```
python manage.py migrate
```

Si se desea cargar los datos de ejemplo:

```
python create_data.py
```

Para iniciar Django:

```
python manage.py runserver
```

La aplicación queda disponible en:

```
http://127.0.0.1:8000/
```

Conclusiones

El proyecto cumple con los principales requisitos planteados para el Entregable Final de Programación IV.

Se implementó un CRUD completo para la entidad **House**, incluyendo identificador único, validaciones y mensajes para las operaciones.

Se desarrollaron cinco reportes utilizando los datos reales almacenados en la base de datos.

Se integró Ollama de forma local mediante el modelo personalizado **casas-qwen2.5**, utilizando Django para proporcionar el contexto real de las propiedades.

También se implementaron restricciones para evitar consultas fuera del dominio y mensajes predefinidos cuando no existe información disponible.

El sistema cuenta con manejo de errores cuando Ollama no está disponible.

Se utilizaron los patrones Strategy y Factory para organizar los reportes.

Además, se implementó un historial persistente de conversaciones utilizando **ChatMessage**.

Finalmente, el proyecto cuenta con **30 pruebas automatizadas, todas aprobadas**, documentación técnica, archivo **README.md**, **DOCUMENTACION.md**, **requirements.txt**, **.env.example** y documentación de las sesiones realizadas con OpenCode.

Referencias oficiales

- Django — Documentación oficial: <https://docs.djangoproject.com/>
- Ollama — Documentación oficial: <https://docs.ollama.com/>
- Ollama — Biblioteca de modelos: <https://ollama.com/library>
- OpenCode — Sitio oficial: <https://opencode.ai/>
- Python — Documentación oficial: <https://docs.python.org/>
- Repositorio base utilizado: <https://github.com/ARAVINDs2002/ContextChat-Context-Aware-Chatbot-for-Detail-Pages-Django-Ollama->